Procedural Generation of Knowledge Graphs via Conceptual Blending of Ontologies

Jade S. Franklin[1], Deborah L. McGuinness[1]

[1]Rensselaer Polytechnic Institute (RPI), 110 8th St, Troy, NY

Abstract

Generative AI (GenAI) has significantly broadened the capabilities of automated content generation, but serious challenges remain in ensuring that generated content abides by the requirements and expectations of users. In particular, the black-box nature of GenAI models make it difficult to control how content is generated, leaving open the risk of generating offensive or nonsensical content. However, placing overly strict guardrails on GenAI output can hamper creativity and overall performance.

We propose a framework and algorithmic approach for semantic content generation: the transparent generation of ideas through conceptual blending of knowledge graphs (KGs) within constraints set by ontologies. Our approach allows for GenAI models to assist in both the creation of constituent KGs and the conversion of these KGs into content, but leverages the inherently semantic nature of KGs to ensure high-level idea generation is transparent. The algorithm is designed to accommodate standard RDF KGs as well as OWL ontologies, and is intended for use at a high level for procedural content generation (PCG). By providing a variety of parameters and options that can be adjusted by the user, the algorithm is intended to be widely applicable in scenarios where procedurally generated algorithms are called for. At the same time, it leverages the existing knowledge sources and semantic technologies of the Semantic Web to address issues in non-semantic rule-based methods and compliment the ongoing research in ML and generative AI approaches to PCG.

Keywords

knowledge graph construction, ontology, conceptual blending, computational creativity, procedural content generation

1. Introduction

Content generation has always been a major component of computer science research, but with the recent expansion in abilities of generative AI, computational creativity has received a new boost in attention. Content generation problems cover a broad domain, encompassing both structured and unstructured text generation, asset generation (visual and audible), and arguably even some decision problems. Past research in content generation has mainly been focused on procedural content generation (PCG): the generation of content according to heuristics, rule-based methods, and other algorithmic procedures. PCG has been extensively studied [1, 2], but mainly in the context of game development: algorithms prioritize fast, low-overhead heuristics that most commonly use noise, mathematical computations, and other non-semantic methods that have been fine-tuned to replicate a desired effect for a specific purpose, but suffer from the same faults as other rule-based methods. Most significantly, the average PCG algorithm is not particularly modular, and adjusting algorithms to apply them to a different domain will in

KGC 2024: The Knowledge Graph Conference, May 06–10, 2024, Cornell Tech NYC

0000–0002–1986–0989 (J. S. Franklin); 0000–0001–7037–4567 (D. L. McGuinness)

© 2024 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

some cases require extensive modifications [3]. It has been proposed that applying semantic techniques to PCG algorithms could help solve some of these issues by using intentionality and meaning as first-level concepts within the PCG algorithm itself [4, 5, 6]. Some of these early results demonstrate the usefulness of applying tools such as knowledge graphs and ontologies in a generative context.

At the same time, the usage of generative AI techniques based on transformer architectures has exploded, and the application of these techniques towards standard PCG problems has begun to break barriers that stopped simpler ML methods in the past [7]. However, issues with non-semantics based approaches remain: lack of transparency means lack of adaptability and modularity, and a focus on simulating results without semantic guidance means knowledge-based approaches to PCG hold the potential to overcome pure ML ones [8]. At the same time, AI introduces some new problems related to reliability: if the AI is capable of producing a broad swathe of content, and reproducing anything it has seen in its training data, then it is difficult to control its output and ensure it won't generate anything objectionable or simply nonsensical. Not imposing enough constraints risks getting wildly out-of-range results, but imposing too many constraints risks hampering the AI so much it defeats the purpose of using it in the first place [9].

We propose a new approach towards semantic procedural generation, designed to both leverage existing semantic technologies and knowledge bases and be flexible enough to adapt to the needs of a variety of domains. The approach is for knowledge graph generation, with a focus on generating novel concepts and ideas by leveraging existing knowledge bases such as ontologies. Other works have noted the power of using knowledge graph transformation for novel idea generation [10], and the potential for applying the cognition operation of conceptual blending to Semantic Web technologies in order to advance computational creativity [11]. Our proposed method builds on these ideas by blending knowledge graphs in an ontology-aware way, in order to procedurally generate KGs that conform to the representation of the chosen structure while creating new ideas.

2. Background

The proposed approach draws on a variety of ideas, which, while well-studied individually, are rarely applied together. In subsection 2.1, we define the process of conceptual blending and discuss its applications in Semantic Web technologies. In subsection 2.2, we highlight related work utilizing ontology-based or conceptual blending approaches to knowledge graph construction.

2.1. Conceptual Blending

Conceptual blending refers to the mental operation and theorized cognitive mechanism proposed by Fauconnier and Turner [12] for the transformation of concepts into novel conceptual spaces. It has been expanded upon [13] and developed into frameworks for artificial intelligence and computational creativity [14] by others, but we have chiefly used the computational framework of conceptual blending put forth by Eppe et al. [15] in the semantic context described by Ławrynowicz [11].

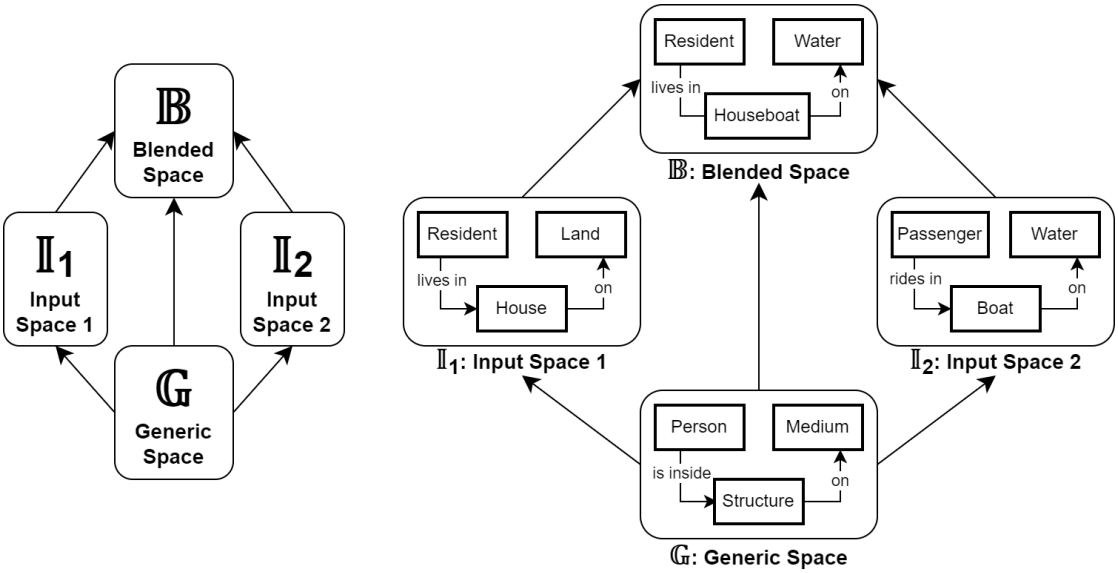

Figure 1: *Left*: The input spaces $\mathbb{I}_1$, $\mathbb{I}_2$ are mapped to a shared "generic space" $\mathbb{G}$, and combined into the "blended space" $\mathbb{B}$.
Right: The common "houseboat" example, as seen in [11] (but slightly modified to better fit the context of ontologies). The output in the blended space is just one of many possible results.

In figure 1, the basic framework of conceptual blending is shown. The two input spaces, $\mathbb{I}_1$ and $\mathbb{I}_2$, are projected to a shared generic space $\mathbb{G}$. Then, based off of their similarities in the generic space, elements of $\mathbb{I}_1$ and $\mathbb{I}_2$ are combined to produce the blended space $\mathbb{B}$. Thus, the operation takes in elements from $\mathbb{I}_1$ and $\mathbb{I}_2$, determines their semantic similarities in $\mathbb{G}$, and uses that to ensure the resulting combination in $\mathbb{B}$ matches the existing structure in $\mathbb{I}_1$ and $\mathbb{I}_2$.

The classic "houseboat" example of conceptual blending is also shown in figure 1 (adapted from [11], but loosely changed to better fit the context of ontologies). In one input space, the concept that a resident lives in a house that belongs on land is shown, whereas in the other input space, the concept that a passenger rides in a boat that floats on water is shown. In this example, the higher-level generic structure shared by both analogies is of a person being inside a structure, and that structure being built on some medium. Therefore, one possible blend is a *houseboat*: a resident lives in a houseboat that floats on water. This is just one possibility: another is of a passenger riding in a *landboat* that resides on land (i.e., a sled).

2.2. Related Work

The application of conceptual blending towards ontologies has been studied extensively by Gangemi et al. [16], who examined usage of Framester with conceptual blending in order to understand metaphors. Conceptual blending applications with ontologies have been developed by Zhang et al. [17], who examined the conceptual blending of ontologies towards mathematical applications, and Shen et al. [18], who proposed the conceptual blending of knowledge graphs for product generation in apparel design. Conceptual blending has also been explored in non-

semantic contexts related to procedural content generation: Sarkar et al. [8] conducted a review of procedural content generation approaches that take a knowledge-first approach, and notes some examples where the authors incorporated conceptual blending.

However, procedural content generation techniques which are designed to leverage existing semantic technologies such as knowledge graphs and ontologies for conceptual blending have not yet been studied in the PCG field. The methods proposed in this paper are intended to build off of individual components of prior research, particularly the general ideas proposed by Ławrynowicz, and leverage them into a more general framework for further testing and analysis across domains.

3. Methods

The proposed method consists of four steps:

1. The generic space $\mathbb{G}$ is identified from superclasses and superproperties shared by $\mathbb{I}_1$ and $\mathbb{I}_2$.
2. Templates are created from nodes in $\mathbb{I}_1$ that share subtypes with $\mathbb{G}$.
3. Candidate nodes and properties are selected from $\mathbb{I}_2$ and matched against the template.
4. Blended concept KGs are created by systematically combining the candidate nodes with the template.

These methods are examined in more detail in the following sections, and an algorithm for knowledge graph generation is provided. The algorithm has been tested to produce the "houseboat" result discussed in section 2, as well as a variety of other examples discussed in Coulson et al. [13].

3.1. Identification of Generic Space

We propose a simple framework for adapting Fauconnier and Turner conceptual blending to usage with knowledge graphs, as shown in figure 2. For each of the nodes and properties in the input spaces $\mathbb{I}_1$, $\mathbb{I}_2$ and the blended space $\mathbb{B}$, a common supertype of all three is present in the generic space $\mathbb{G}$. In addition, the structure of these nodes and properties are analogous to one another in all four spaces: for a given triple (s, p, o) in one space, a corresponding triple (s', p', o') exists in each of the other three spaces, where the pair s,s', the pair p,p', and the pair o,o' share the type or supertype S, P, or O, respectively[1]. In this context, we do not impose any requirements for the nodes and properties in each space to be separate; for example, any or all of s, s', and S could be equivalent or different from one another.

Note that this framework provides a certain degree of flexibility: two nodes a_1 and a_2 could have many shared types and supertypes in $\mathbb{G}$, from the very low-level to the very high-level. Different implementations of this method can handle this differently: for example, removing certain supertypes from $\mathbb{G}$ as too high-level, requiring certain types be included in $\mathbb{G}$, or enforcing minimum or maximum bounds on the number of levels shared by a_1 and a_2.

[1]Depending on the KG, it may not make sense to require (S,P,O) to exist in $\mathbb{G}$. Instead, the requirement should be that P may have S as a domain and O as a range.

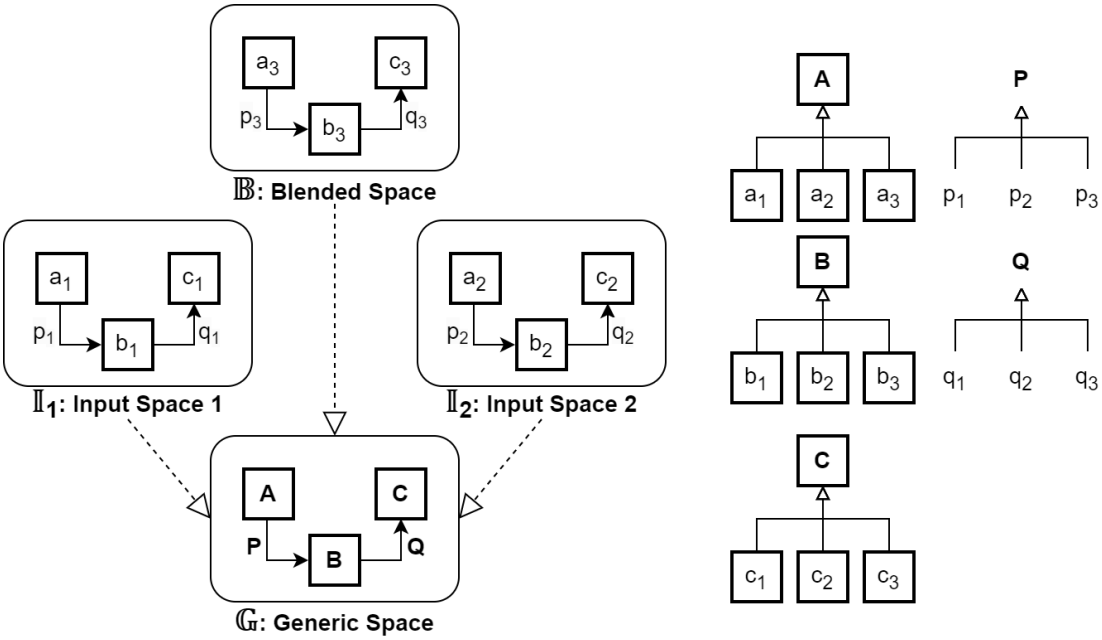

Figure 2: Each of $\mathbb{I}_1$, $\mathbb{I}_2$, and $\mathbb{B}$ consist of classes and properties that are subtypes of $\mathbb{G}$ (the subtype relation represented by unfilled arrows). Note that a_3,p_3, etc. might be distinct from a_1,a_2,p_1,p_2, etc., but not necessarily.

Differences in these additional constraints may be made between the different input spaces and the output/blended space as needed. Additionally, equivalency and/or non-equivalency requirements can be enforced as well: it could be required that all of a_1, a_2, and a_3 must correspond to different instances, or it could be required that a_3 is one of either a_1 or a_2. All of these constraints can easily be added via adjustments to the constraints in SPARQL queries or description logic queries used to retrieve the generic space during implementation.

For compatibility with OWL ontologies, the framework can be slightly adjusted: instead of requiring the generic space $\mathbb{G}$ to contain the triple (S,P,O), it can be required that the class S is a subtype of a existential restriction on property P to have a range of some value from O. In description logic terms, each of s_1, s_2, and s_3 should be returned from the DL query `S and P some O`.

3.2. Blending via Templates and Candidates

Once the generic space has been identified, it used as the basis for combining elements of $\mathbb{I}_1$ and $\mathbb{I}_2$. Figure 3 shows the process of combining templates with candidates to perform the conceptual blending. Broadly, we use *template* to describe a KG subset of $\mathbb{I}_1$ with some nodes and properties left unspecified as *empty* elements: missing elements with constraints from $\mathbb{G}$ imposed upon them that must be "filled in" with nodes and properties from $\mathbb{I}_2$. A set of nodes and properties from $\mathbb{I}_2$ that are to be matched against the constraints set by the template are referred to as a textitcandidate: each node and property in the candidate corresponds to an

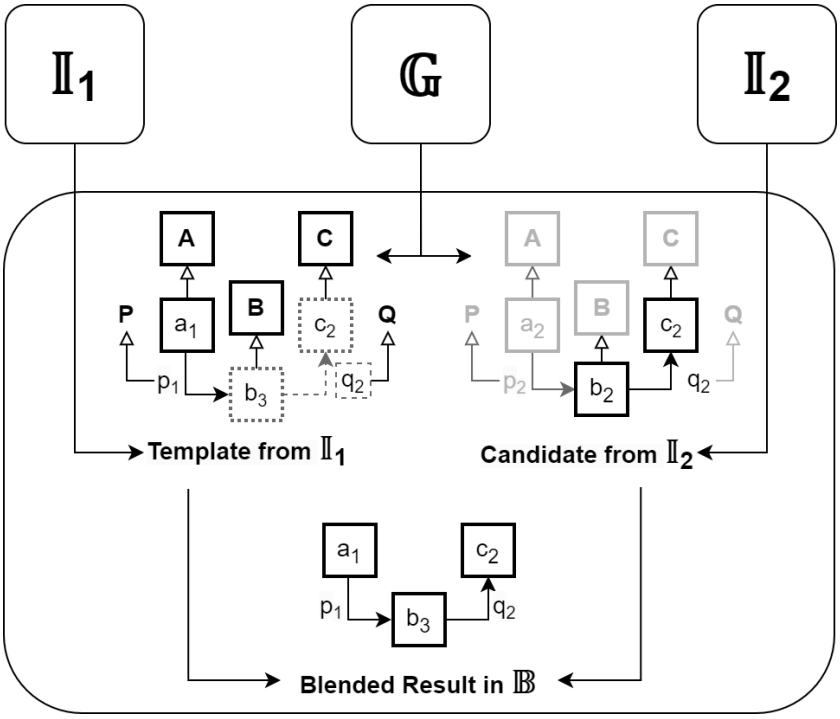

Figure 3: On the left, a template is created from $\mathbb{I}_1$ and $\mathbb{G}$, where boxes and arrows drawn with gray dashed lines represent the empty nodes and edges to be filled by valid candidates. On the right, candidates are selected from $\mathbb{I}_2$ to attempt to fill in the empty spots in the template; the light gray nodes and properties represent the constraints imposed by the template and $\mathbb{G}$. At the bottom, the blended result formed from completing the template with the candidate is shown; note that a new node b_3 has been created at the direct intersection of the template with to avoid collisions with nodes b_1 and b_2.

empty node or empty property in the template, and must meet all of the constraints imposed on the empty node or property in order to create a valid result.

Representation of templates in RDF is possible but not required (they can also be intrinsically represented via constraints on variables in SPARQL queries, for example). If representation in RDF is desired, empty nodes could be represented as blank nodes, but to avoid confusion with other blank nodes that may be a part of the constraints imposed by the template we encourage users to give them a custom type or other property. We do not provide an explicit type, as empty nodes for different conceptual blending processes should have the option of being represented by different types, so as to allow the template creation process to be performed upon other (previously generated) templates. Representation of empty properties cannot be done in the exact same way, as RDF does not support blank edges; if the only constraint imposed upon the empty property is a superproperty, then the superproperty itself could be used to represent an empty edge when placed between two empty nodes. If a greater degree of flexibility is required, reification techniques or OWL property restrictions could be used instead.

3.2.1. Template Creation

The first step in template creation is to select a generic structure from $\mathbb{G}$ to use for the structure of elements projected from $\mathbb{I}_1$, $\mathbb{I}_2$, and $\mathbb{B}$. The methods described previously for identifying a generic space will find all possible generic overlap between $\mathbb{I}_1$ and $\mathbb{I}_2$, and may result in a significantly larger $\mathbb{G}$ than the examples discussed in section 2.1; therefore, only a connected subgraph of $\mathbb{G}$ (a generic structure) will be used instead. One method is to iteratively select every component of $\mathbb{G}$ to use as a generic structure for a new template, but a generic structure that is too large will impose too many constraints upon the input spaces. Enforcing a maximum size for generic structures may be ideal in this case, or relaxing the requirements for candidate matching as described in section 3.2.2. The other option is to use a human in the loop: this could be either manual selection of generic structures according to the needs of the use case, or selection of generic structures which contain certain nodes marked as desirable (again, a minimum bound on subgraph size could be useful in this case). Either way, for each generic structure that has been selected, the rest of $\mathbb{G}$ can be ignored when generating and matching templates for this structure—in other words, create a pruned $\mathbb{G}_{\mathbb{i}}$ such that is equivalent to the generic structure.

To create a template from $\mathbb{G}_{\mathbb{i}}$ and $\mathbb{I}_{\mathbb{1}}$, select a node N_0 from $\mathbb{G}_{\mathbb{i}}$ to use as a starting point. This may be selected automatically (for example, selecting as a starting point each node in $\mathbb{G}_{\mathbb{i}}$ from which every other node in $\mathbb{G}_{\mathbb{i}}$ can be reached), or it may be aided via a human in the loop, such as selecting a starting node marked as desirable as described above. For each node $n_0 \subseteq N_0 \in \mathbb{I}_{\mathbb{1}}$,[2] find every triple $(n_j, p_j, n_{j+1}) \in \mathbb{I}_{\mathbb{0}}$ such that, for each triple $(N_j, P_j, N_{j+1}) \in \mathbb{G}_{\mathbb{i}}$, every $n_j \subseteq N_j$ and $p_i \subseteq P_i$. This confirms that N_0 is indeed part of the structure matching $\mathbb{G}_{\mathbb{i}}$. Then, convert this structure into a template by selecting some "swap node" in the structure n_k, where $k > 0$ and $N_K \in \mathbb{G}_{\mathbb{i}}$. This node is to be the first empty node in the template, and has the constraint. $n_k \subseteq N_K$. Then, if $N_K + 1 \in \mathbb{G}_{\mathbb{i}}$, add the constraints (n_k, p_k, n_{k+1}) and $p_k \subseteq P_K$, add p_k and n_k to the template as an empty property and node, respectively, and repeat until all nodes have been added (or a preselected stop size m or stop node n_m is reached). The end result is a template with k nodes from $\mathbb{I}_{\mathbb{1}}$, and $m - k$ empty nodes to be filled from $\mathbb{I}_2$.

Many templates can be created from the same starting node n_0 by selecting different values for k, and many more combinations may be created from selecting different n_0 nodes from $\mathbb{I}_{\mathbb{0}}$, different N_0 from $\mathbb{G}_{\mathbb{i}}$, and different $\mathbb{G}_{\mathbb{i}} \subset \mathbb{G}$. Therefore, pre-selecting starting nodes from $\mathbb{I}_{\mathbb{0}}$ from or even entire templates that are known to lead to desirable results may be most efficient. If scalability is a concern, preselecting desirable starting generic nodes N_0 from $\mathbb{G}_{\mathbb{i}}$, or nodes within $\mathbb{G}$ to additionally use for generic structure selection, can lead to a middle ground between manual control and large scale of results. Similarly, selection of swap nodes $n_k \in \mathbb{I}_{\mathbb{1}}$ and stop nodes $n_m \in \mathbb{I}_{\mathbb{1}}$ can offer fine-grained control, whereas selection of generic swap nodes and stop nodes $N_k, N_m \in \mathbb{G}$, where $n_k \subseteq N_k$ and $n_m \subseteq N_m$, can present a middle ground.

The template is explicitly created using elements from $\mathbb{I}_1$ as a base, allowing the user to exercise control over which input space should be used as a starting point in the conceptual blending process. This may be relevant if $\mathbb{G}$ contains properties without an inverse; in that case, elements from $\mathbb{I}_{\mathbb{1}}$ will tend to be subjects, and elements from $\mathbb{I}_2$ will tend to be objects.

[2] $a \subseteq A$ represents a is the type or subtype of A

3.2.2. Candidate Matching and Ranking Results

Candidate matching uses much the same algorithm as template generation, but with far less variables as the template size and shape has already been created. For a given candidate structure $n_0, ..., n_m \in \mathbb{I}_2$, check each node n_j and property p_j and verify whether the template constraints have been met (i.e., $n_j \subseteq N_j$, $p_j \subseteq P_j$, and if $N_{j+1} \in \mathbb{G}_{\mathbb{i}}$, then $\{(n_j, n_j, n_{j+1})\} \subseteq \mathbb{I}_2$ and $\{(N_j, P_j, N_{j+1})\} \subseteq \mathbb{G}_{\mathbb{i}}$.[3] Once the candidate has been verified, use the combined set of nodes as the blended result: for $0 < j < k$ (non-empty nodes/nodes before the swap), add nodes $n_j \in \mathbb{I}_1$ to $\mathbb{B}$, and for $k < j < n$ (empty nodes/nodes after the swap), add nodes $n_j \in \mathbb{I}_2$ to $\mathbb{B}$. For the nodes at $j = k$ (the swap nodes), adding the corresponding node from either $\mathbb{I}_1$ or $\mathbb{I}_2$ could cause a conflict with existing knowledge graphs. Therefore, it is recommended to create a new node $n_k \in \mathbb{B}$ and selectively add properties to it from $n_k \in \mathbb{I}_1$,$n_k \in \mathbb{I}_2$, as long as they won't cause a conflict. This corresponds to the newly generated concept "houseboat" in the example in figure 1.

There are some optional parameters for the candidate matching step that can be controlled, and may also be used to rank results. The simplest of these is to simply relax the requirements for candidate matching—rather than require *all* constraints to be satisfied, set a minimum bound on the number of constraints, and rank the results by percentage satisfied (or a more advanced algorithm for semantic similarity). Similarly, additional constraints or relaxations such as those mentioned in section 3.1 can be used, both for filtering and ranking: distance from ideal size of concept blend result, distance from ideal number of levels of subtype, ascending or descending in order of percentage of nodes with exact identity matches, number of nodes that meet additional desirability requirements, and so on.

One additional option for adjusting the matching algorithm itself is to enable property path search: for a generic structure with the triple pattern $((N_j, P_j, N_{j+1}))$, allow x additional properties between $n_j \subseteq N_j$ and $n_{j+x+1} \subseteq N_{j+1}$, as long as each property between p_j and p_{j+x} is a subtype of P_i. This may help account for differences between structure sizes in input spaces, and can be used as an additional pattern in ranking (length of property path).

4. Discussion

The proposed algorithm is a novel approach for procedural content generation using knowledge graphs and conceptual blending. It has been implemented and tested in Python, and applied towards the structured extraction of knowledge from research papers. Research in its use in the procedural generation of game elements is ongoing, but further evaluation is needed to understand the strengths and weaknesses of the approach. At present, one possible area of concern is scalability: a great variety of options and parameters were introduced in the algorithm to increase or decrease the quantity, variety, and quality of results, but extensive testing is needed to determine the impact of each of these factors and whether too much manual labor is required to generate optimal results. In theory, the adaptablity of the proposed method to a variety of ontologies and other KG-based knowledge sources mitigates this, and gives it a semantic edge over other approaches, but it will need evaluation across a variety of ontologies (at

[3] Or verify that N_i is a valid domain domain of P_i and N_{i+1} is a valid range, if using the domain/range option.

present, current implementations have been using Framester [19] and DBPedia [20]). Another area of concern is broader applicability: knowledge graphs have been demonstrated in a variety of procedural generation applications, but ultimately knowledge graph generation requires systems that can use these knowledge graphs to good effect in order to be useful.

However, with the emergence of large language models, it may be that knowledge graph generation is more important than ever. Large language models have been shown to reliably be able to utilize knowledge graphs for further generative activities[21, 22, 23], and there are a variety of promising applications for knowledge graph generation with generative AI techniques [24]. Although the application of conceptual blending to semantic technologies and the application of semantic technologies to PCG have each individually been studied, there has not yet been a work studying a procedural content generation application for conceptual blending of ontologies and knowledge graphs. This work is intended as a small step towards further adoption of semantic technologies and computational creativity techniques in new fields, and may help bridge the gap between the semantic and the generative AI approaches to concept generation.

Acknowledgments

Thank you to Mav Modi, John Slowik, and Jamie McCusker for their support and encouragement for this work.

References

[1] K. Compton, M. Mateas, A generative framework of generativity, in: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, volume 13, 2017, pp. 41–48.

[2] R. Khaled, M. J. Nelson, P. Barr, Design metaphors for procedural content generation in games, in: Proceedings of the SIGCHI conference on human factors in computing systems, 2013, pp. 1509–1518.

[3] R. Smelik, T. Tutenel, K. J. De Kraker, R. Bidarra, Integrating procedural generation and manual editing of virtual worlds, in: Proceedings of the 2010 Workshop on Procedural Content Generation in Games, 2010, pp. 1–8.

[4] T. Tutenel, R. Smelik, R. Bidarra, K. J. de Kraker, Using semantics to improve the design of game worlds, in: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, volume 5, 2009, pp. 100–105.

[5] R. Lopes, Gameplay semantics for the adaptive generation of game worlds, Citeseer, 2014.

[6] J. T. Balint, R. Bidarra, Procedural generation of narrative worlds, IEEE Transactions on Games 15 (2022) 262–272.

[7] J. Liu, S. Snodgrass, A. Khalifa, S. Risi, G. N. Yannakakis, J. Togelius, Deep learning for procedural content generation, Neural Computing and Applications 33 (2021) 19–37.

[8] A. Sarkar, M. Guzdial, S. Snodgrass, A. Summerville, T. Machado, G. Smith, Procedural content generation via knowledge transformation (pcg-kt), IEEE Transactions on Games (2023).

[9] G. Marcus, Deep learning is hitting a wall, Nautilus, Accessed (2022) 03–11.
[10] D. Zhou, B. Zhou, Z. Zheng, A. Soylu, G. Cheng, E. Jimenez-Ruiz, E. V. Kostylev, E. Kharlamov, Ontology reshaping for knowledge graph construction: applied on bosch welding case, in: International Semantic Web Conference, Springer, 2022, pp. 770–790.
[11] A. Ławrynowicz, Creative AI: A new avenue for the semantic web?, Semantic Web 11 (2020) 69–78.
[12] G. Fauconnier, M. Turner, Conceptual integration networks, Cognitive science 22 (1998) 133–187.
[13] S. Coulson, T. Oakley, Blending basics, Cognitive Linguistics 11 (2001) 175–196. doi:doi:10.1515/cogl.2001.014.
[14] F. Câmara Pereira, Creativity and artificial intelligence: a conceptual blending approach, Mouton de Gruyter, 2007.
[15] M. Eppe, E. Maclean, R. Confalonieri, O. Kutz, M. Schorlemmer, E. Plaza, K.-U. Kühnberger, A computational framework for conceptual blending, Artificial Intelligence 256 (2018) 105–129.
[16] A. Gangemi, M. Alam, V. Presutti, Amnestic forgery: An ontology of conceptual metaphors, arXiv preprint arXiv:1805.12115 (2018).
[17] Q. L. Chao Zhang, Guanghui Zhou, F. Chang, Graph-based knowledge reuse for supporting knowledge-driven decision-making in new product development, International Journal of Production Research 55 (2017) 7187–7203. doi:10.1080/00207543.2017.1351643.
[18] X. Shen, X. Li, B. Zhou, Y. Jiang, J. Bao, Dynamic knowledge modeling and fusion method for custom apparel production process based on knowledge graph, Advanced Engineering Informatics 55 (2023) 101880.
[19] A. Gangemi, M. Alam, L. Asprino, V. Presutti, D. R. Recupero, Framester: A wide coverage linguistic linked data hub, in: Knowledge Engineering and Knowledge Management: 20th International Conference, EKAW 2016, Bologna, Italy, November 19-23, 2016, Proceedings 20, Springer, 2016, pp. 239–254.
[20] S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, Z. Ives, Dbpedia: A nucleus for a web of open data, in: international semantic web conference, Springer, 2007, pp. 722–735.
[21] H. Abu-Rasheed, C. Weber, M. Fathi, Knowledge graphs as context sources for llm-based explanations of learning recommendations, arXiv preprint arXiv:2403.03008 (2024).
[22] C. Feng, X. Zhang, Z. Fei, Knowledge solver: Teaching llms to search for domain knowledge from knowledge graphs, arXiv preprint arXiv:2309.03118 (2023).
[23] J. Zhang, Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt, arXiv preprint arXiv:2304.11116 (2023).
[24] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, X. Wu, Unifying large language models and knowledge graphs: A roadmap, IEEE Transactions on Knowledge and Data Engineering (2024).